LAPORAN PRAKTIKUM
DESAIN WEB
MODUL 2

Dosen Pengampu: Dede Irawan M.Kom



Disusun oleh :

Nama	**:**	**Lea Iqbal Meicca Putra**
NIM	**:**	**20250910087**
Kelas	**:**	**SINFC-2025-02**

SISTEM INFORMASI
FAKULTAS ILMU KOMPUTER
UNIVERSITAS KUNINGAN
2026

KATA PENGANTAR

Laporan ini dibuat untuk pelaksanaan praktikum Modul 2 mengenai Git yang merupakan bagian dari mata kuliah Praktikum Desain Web dengan dosen pengampu Dede Irawan M. Kom. Praktikum ini bertujuan agar mahasiswa memahami konsep version control serta mampu menggunakan perintah dasar Git dan GitHub untuk mengelola proyek. Dalam laporan ini, saya menuliskan jawaban pre-test, hasil praktikum, jawaban post-test, serta tugas beserta analisis sederhana dari hasilnya.

Saya menyadari bahwa dalam penyusunan laporan ini masih terdapat kekurangan karena keterbatasan pengetahuan dan kemampuan saya. Oleh karena itu, saya sangat menghargai kritik dan saran untuk menyempurnakan laporan ini. Terima kasih saya ucapkan kepada Dede Irawan M. Kom serta asisten praktikum yang telah memberikan bimbingan selama praktikum. Semoga laporan ini dapat memberikan manfaat dan memenuhi tugas sesuai harapan.

Kuningan, 1 November 2026

Lea Iqbal Meicca Putra

DAFTAR ISI

KATA PENGANTAR i
DAFTAR ISI iii
BAB I 4
PRETEST **4**
BAB II 5
PEMBAHASAN **5**
BAB III 7
POSTEST **7**
BAB IV 8
TUGAS **8**
BAB V 9
KESIMPULAN **9**

BAB I

PRETEST

1. Apa tujuan dari menggunakan version control dalam pengembangan perangkat lunak?

   Tujuan version control adalah mencatat dan mengelola setiap perubahan pada kode program dari waktu ke waktu. Dengan begitu pengembang dapat melihat riwayat perubahan, mengetahui siapa mengubah apa dan kapan, serta mengembalikan proyek ke versi sebelumnya jika terjadi kesalahan. Version control juga memungkinkan beberapa orang mengerjakan proyek yang sama secara bersamaan tanpa saling menimpa pekerjaan.

2. Jelaskan perbedaan antara git add dan git commit!

   Perintah git add menaruh perubahan pada file ke staging area, yaitu tempat persiapan untuk menentukan perubahan mana yang akan direkam. Perintah ini belum menyimpan apa pun ke riwayat repository. Perintah git commit merekam seluruh isi staging area ke repository sebagai satu versi permanen yang diberi pesan dan kode unik. Jadi git add memilih perubahan, dan git commit menyimpannya.

3. Bagaimana cara membuat branch baru di Git?

   Branch baru dibuat dengan perintah git branch nama-branch, lalu pindah ke branch itu dengan git checkout nama-branch. Keduanya dapat digabung dengan satu perintah, yaitu git checkout -b nama-branch, atau git switch -c nama-branch pada Git versi baru.

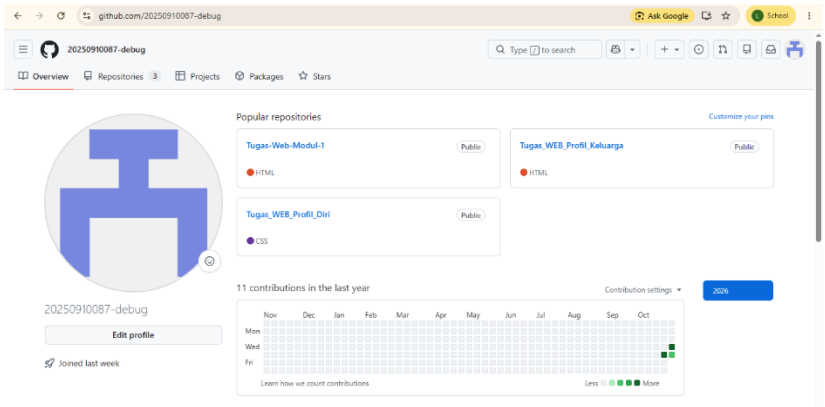
BAB II

PEMBAHASAN

PRAKTIKUM

1. MEMBUAT AKUN GITHUB

   Setelah mengisi e-mail, kata sandi, dan username, saya menyelesaikan teka-teki verifikasi, lalu memasukkan kode yang dikirim ke e-mail. Jika benar, muncul halaman Sign in dengan pesan bahwa akun berhasil dibuat, lalu saya login dan memilih paket Free.



2. INSTALASI GIT

   Perintah git --version menampilkan nomor versi Git, misalnya “git version 2.43.0”, yang menandakan instalasi berhasil. Perintah git config untuk nama dan e-mail tidak menampilkan apa pun jika berhasil, tetapi data itu akan tercatat pada setiap commit.

3. MENGELOLA PROYEK DENGAN GIT

   Perintah git init membuat repository kosong dan folder tersembunyi .git. Sebelum git add, perintah git status menampilkan file berstatus untracked. Setelah git add, statusnya menjadi “Changes to be committed” dengan keterangan new file. Perintah git commit menampilkan kode commit dan ringkasan seperti “1 file changed, 21 insertions(+)”, yang berarti file sudah tersimpan di riwayat.

4. MENGHUBUNGKAN KE GITHUB DAN PUSH

   Perintah git remote add origin menghubungkan repository lokal ke GitHub tanpa menampilkan apa pun, dan git branch -M main mengganti

nama branch menjadi main. Perintah git push -u origin main menampilkan proses pengiriman data dan pesan bahwa branch main terbentuk di remote. Setelah halaman repository dimuat ulang, file beserta pesan commitnya tampil di GitHub.

BAB III

POSTEST

1. Apa fungsi dari git merge dalam workflow Git?

   Perintah git merge menggabungkan perubahan dari satu branch ke branch yang sedang aktif. Dalam alur kerja Git, fitur baru dikerjakan di branch terpisah, dan setelah selesai serta diuji, hasilnya digabungkan ke branch utama dengan merge. Pada percobaan, penggabungan branch fitur ke main menampilkan kata “Fast-forward” beserta daftar file yang bertambah, yang artinya main tidak berubah sejak branch dibuat sehingga Git cukup menggeser posisi main ke commit terbaru.

2. Jelaskan cara mengatasi konflik saat melakukan merge!

   Konflik terjadi ketika dua branch mengubah bagian yang sama dari file yang sama, sehingga Git tidak bisa memilih sendiri. Pada percobaan, Git menampilkan “CONFLICT (content): Merge conflict” dan status “You have unmerged paths” dengan keterangan “both modified” pada file yang bermasalah. Di dalam file, Git menandai konflik dengan tiga penanda: bagian di atas tanda sama dengan adalah versi branch aktif (HEAD), dan bagian di bawahnya adalah versi branch yang digabungkan. Cara mengatasinya adalah membuka file tersebut, memilih versi yang dipakai atau menggabungkan keduanya, lalu menghapus semua baris penanda. Setelah itu file dimasukkan kembali dengan git add dan diselesaikan dengan git commit. Jika ingin membatalkan penggabungan, gunakan git merge --abort.

3. Tulis langkah-langkah untuk menambahkan file baru ke repository Git dan melakukan commit.

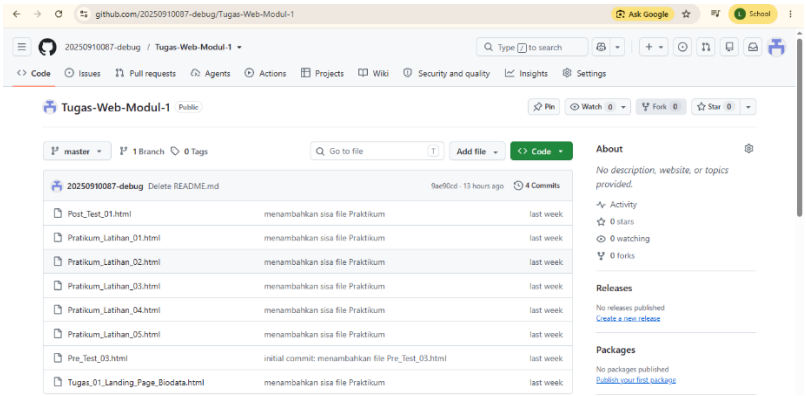
   Pertama, buat atau salin file baru ke folder proyek. Kedua, jalankan git status untuk memastikan file terdeteksi sebagai untracked. Ketiga, jalankan git add nama-file untuk memasukkannya ke staging area. Keempat, jalankan git commit -m "pesan commit" untuk merekamnya. Jika ingin mengirimnya ke GitHub, tambahkan langkah kelima, yaitu git push.

BAB IV

TUGAS

1. Buatlah repository Git untuk proyek web dan dokumentasikan proses pembuatan, commit, dan push ke GitHub.

   Buka folder kerja di Visual Studio Code, lalu jalankan git init. Masukkan file proyek dengan git add, rekam dengan git commit -m, hubungkan ke repository GitHub dengan git remote add origin, lalu kirim dengan git push -u origin main. Hasilnya, file dan riwayat commit tampil di halaman repository GitHub.



2. Rancanglah workflow Git yang melibatkan branch untuk fitur baru dan integrasi dengan branch utama.

   Branch main dijaga tetap stabil. Fitur baru dikerjakan di branch terpisah yang dibuat dengan git checkout -b fitur-kontak, dengan git add dan git commit berulang di sana. Setelah selesai dan diuji, pindah ke main dengan git checkout main, lalu gabungkan dengan git merge fitur-kontak dan kirim hasilnya dengan git push. Branch fitur yang sudah selesai dapat dihapus dengan git branch -d.

3. Simpan seluruh praktikum pada modul sebelumnya ke dalam repository Git yang sudah dibuat.

   Pastikan folder Modul_01 beserta semua filenya berada di dalam folder repository. Jalankan git add ., lalu git commit -m "Menambahkan seluruh praktikum Modul 1", kemudian git push. Hasilnya, folder Modul_01 dengan semua filenya tampil di halaman repository GitHub.

BAB V

KESIMPULAN

Berdasarkan praktikum Modul 2 tentang Git, dapat disimpulkan bahwa Git adalah sistem kontrol versi yang mencatat setiap perubahan pada file proyek, sehingga pengembang dapat melihat riwayat perubahan, kembali ke versi sebelumnya, dan bekerja bersama tanpa saling menimpa pekerjaan. Git berjalan di komputer lokal, sedangkan GitHub berfungsi menyimpan repository di internet agar dapat diakses dan dibagikan. Alur kerja dasarnya dimulai dari git init untuk membuat repository, git add untuk memilih perubahan ke staging area, git commit untuk merekamnya sebagai versi permanen, lalu git push untuk mengirimnya ke GitHub. Perintah git status membantu memantau kondisi file pada setiap tahap.

Praktikum ini juga menunjukkan bahwa branch memungkinkan pengembangan fitur baru tanpa mengganggu branch utama, dan git merge menggabungkan hasilnya setelah selesai. Jika dua branch mengubah bagian yang sama, terjadi konflik yang harus diselesaikan secara manual dengan memilih isi yang dipakai, menghapus penanda konflik, lalu melakukan commit ulang. Melalui tugas, saya berhasil membuat repository, mencatat perubahan lewat commit, mengirimnya ke GitHub, dan menyimpan seluruh praktikum Modul 1 ke dalamnya. Dengan demikian, tujuan Modul 2, yaitu memahami konsep version control, memakai perintah dasar Git, dan mengelola proyek dengan GitHub, telah tercapai.